

Feature Selection Methodology

High-Dimensional Clinical Data Pipeline

1. Overview

This document describes the feature selection methods implemented for high-dimensional clinical data analysis. The pipeline supports both classification and regression tasks and is designed for datasets where features outnumber samples (HDLSS scenarios).

Two primary methods are implemented: **GRACES** (graph-based) and **DeepFS** (autoencoder-based). Both methods are data-agnostic and work with any tabular clinical data.

2. GRACES (Graph Convolutional Network Feature Selection)

2.1 Method

GRACES uses a GraphSAGE neural network to learn feature importance through gradient-based saliency. Samples are connected via cosine similarity edges, allowing the model to capture sample relationships during feature selection.

2.2 Algorithm Steps

1. Standardize input features (zero mean, unit variance)
2. Build sample similarity graph using cosine similarity
3. Train GraphSAGE network on classification/regression task
4. Compute gradient-based saliency scores for each feature
5. Combine gradients with F-test scores (optional)
6. Select top-k features by importance score

2.3 Graph Construction

```
similarity_matrix = |cosine_similarity(samples)|
threshold = quantile(similarity, alpha=0.95)
edges = similarity_matrix >= threshold
```

2.4 Saliency Computation (Regression)

```
saliency = mean(|grad(loss) w.r.t. input|, dim=samples)
selected = top_k(saliency, k=n_features)
```

2.5 Forward Selection (Classification)

```
S = [bias_term]
for i in range(n_features):
    train GNN on S
    grad = average_gradient(dropout_ensemble)
    next_feature = argmax(grad_norm + f_correct * f_scores)
    S.append(next_feature)
```

2.6 Parameters

```
n_features      = 100 (classification) / 50 (regression)
hidden_size     = [64, 32]
epochs          = 50 (classification) / 100 (regression)
alpha           = 0.95 (edge threshold percentile)
learning_rate   = 0.001
batch_size      = 16 (classification) / 8 (regression)
n_dropouts      = 10 (classification only)
dropout_prob    = 0.5 (classification only)
```

2.7 Reference

Chen et al. *Graph Convolutional Network-based Feature Selection for High-dimensional and Low-sample Size Data* (arXiv:2211.14144)

3. DeepFS (Deep Feature Screening)

3.1 Method

DeepFS uses a supervised autoencoder to learn a latent representation of the data, then ranks features by their Rank Distance Correlation (RdCorr) with the learned embedding. This captures non-linear relationships without assuming feature distributions.

3.2 Algorithm Steps

- 1. Standardize input features
- 2. Train supervised autoencoder (reconstruction + prediction loss)
- 3. Extract latent embeddings from encoder
- 4. Compute RdCorr between each feature and latent embedding
- 5. Select top-k features by RdCorr score

3.3 Supervised Autoencoder Architecture

```
Encoder: Linear(input_dim, 100) -> ReLU -> Linear(100, latent_dim)
Decoder: Linear(latent_dim, 100) -> ReLU -> Linear(100, input_dim)
Head:    Linear(latent_dim, n_classes) [classification]
         Linear(latent_dim, 1) [regression]
```

3.4 Loss Function

```
loss = loss_supervised + lambda_reg * loss_reconstruction
```

```
Classification: loss_supervised = CrossEntropyLoss
Regression:     loss_supervised = MSELoss
```

3.5 Rank Distance Correlation

1. Normalize features to $[0, 1]$
2. Compute multivariate ranks using Sobol sequences
3.
$$\text{RdCov}(X, Y) = \frac{\sum(A*B)}{n^2} + \frac{\sum(A)*\sum(B)}{n^4} - \frac{2*\sum(\text{rowsum}(A)*\text{rowsum}(B))}{n^3}$$
where A, B are pairwise distance matrices
4.
$$\text{RdCorr} = \text{RdCov}(X, Y) / \sqrt{\text{RdCov}(X, X) * \text{RdCov}(Y, Y)}$$

3.6 Parameters

```
n_features   = 100 (classification) / 50 (regression)
latent_dim   = 15
epochs       = 500
lr           = 0.001
lambda_reg   = 0.01
```

3.7 Reference

Li et al. *Deep Feature Screening: Feature Selection for Ultra High-Dimensional Data via Deep Neural Networks* (arXiv:2204.01682)

4. Data Preprocessing

4.1 Feature Standardization

All features are standardized using sklearn StandardScaler before feature selection:

```
X_scaled = (X - mean(X)) / std(X)
```

4.2 Data Reduction (Optional)

For simulating low-sample scenarios, random samples can be removed:

```
n_keep = n_original * (1 - reduction_pct)
indices = random.choice(n_original, n_keep, replace=False)
```

4.3 SMOTE (Classification)

For imbalanced classification, SMOTE generates synthetic minority samples:

```
smote = SMOTE(k_neighbors=5)
X_balanced, y_balanced = smote.fit_resample(X, y)
```

4.4 SMOGN (Regression)

For imbalanced regression, SMOGN handles rare target values. Key feature: samples are only generated within the same tail (low or high) to prevent cross-tail pollution.

```
Rare regions: y <= 5th percentile OR y >= 95th percentile
Synthetic cap: smogn_limit * n_rare (default: 100% growth max)
Interpolation: only between samples in same tail
```

5. Pipeline Integration

5.1 Execution Order

1. Load data (X, y)
2. Train/test split
3. [Simulator] Data reduction (optional)
4. [Simulator] SMOTE/SMOGN augmentation (optional)
5. [Toolbox] Feature selection (GRACES or DeepFS)
6. [Classifier/Regressor] Model training on selected features
7. [Evaluation] Compute metrics

5.2 Usage

Classification:

```
python main.py --data dataset.csv --target label \
  --selector graces --features 100 --smote
```

Regression:

```
python main.py --data dataset.csv --target value \
  --selector deepfs --features 50 --smogn
```

6. Method Selection Guide

Use GRACES when:

- Sample relationships matter (similar patients share features)
- Forward selection provides interpretable feature ordering
- F-test integration desired for statistical rigor

Use DeepFS when:

- Non-linear feature relationships exist
- Distribution-free correlation preferred
- Ultra-high-dimensional data (faster per-feature computation)

7. Output

Feature selection returns indices of selected features. The selector object stores:

```
selector.selected_indices_ # Array of selected feature indices
selector.feature_scores_   # Importance scores (DeepFS only)
selector.is_fitted_        # Boolean fit status
```

To apply selection:

```
selector.fit(X_train, y_train)
X_train_selected = selector.transform(X_train)
X_test_selected = selector.transform(X_test)
```